FAIR-X: A Threshold-Transfer Evaluation Contract for Behavioral Malware Detection under Distribution Shift

Stéphane Gaël R. Ekodeck, *Vivien Beyala* Kamgang, *Serge Alain* Ebélé, *Ulrich Joel* Atchiengang Ngueyep, *Stéphane Willy* Mossebo Tcheunteu, *Damaris Belle* M. Fotso, *Arthur Ulrich* Ewane and *Julius Marcellin* Nkenlifack, *Member, IEEE,*

Abstract—Behavioral malware detectors are often compared by ranking metrics or thresholded scores without proving that their validation-derived operating points remain dependable under security-data shift. We introduce FAIR-X, a threshold-transfer evaluation contract for behavioral security datasets, and instantiate it as FAIR-BETH on process-level BETH. FAIR-X fixes data access, label exclusion, feature-vocabulary fitting, calibration, threshold selection, baseline parity, uncertainty reporting, and claim-boundary audits before final testing. It also defines diagnostic gaps that test whether score fusion adds deployable value beyond strong non-hybrid baselines under identical threshold policies. BETH is a demanding instantiation: after process aggregation, its official training and validation files contain no malicious processes, whereas 121 of 198 official-test processes are malicious. Under a common validation-FPR policy, TabRF reaches an F1 score of 0.7210, RF-500 reaches 0.7043, and MetaRF reaches 0.6756, so score fusion provides no evidence of improvement over direct tabular learning. A repeated five-fold audit fixes 50 thresholds from all 29 development positives before official-test access; their median is 0.006, yet the locked 5% development-FPR threshold produces 32.47% test FPR and F1 0.6964. This separates threshold sampling stability from threshold transfer. Applying the same contract to MalBehavD-V1 yields repeated-split median F1 0.9533. FAIR-X therefore provides a reproducible methodology for distinguishing ranking quality, deployable decisions, and unsupported architectural claims under distribution shift.

Index Terms—Behavioral malware detection, malicious-process detection, dependable security evaluation, threshold transfer, distribution shift, calibration, BETH dataset, MalBehavD-V1, Random Forest, GRU autoencoder.

1 Introduction

Endpoint malware changes faster than static signatures. Payloads can be repacked, recompiled, or launched through different intermediaries, which limits the durability of hashes and portable-executable features [1], [2]. Behavioral detection addresses this limitation by observing what processes do: create files, spawn children, access networks, return errors, and produce temporal event patterns. Its practical value, however, depends on a systems question that is often left implicit: when can a behavioral score be converted into a dependable alert under class imbalance and environmental change?

That score-to-decision conversion is a recurring weakness in security-machine-learning evaluation. AUC and Average Precision measure ranking, not the deployed alerting decision. F1, precision, and recall depend on a threshold whose validation data, objective, and timing are not always comparable across models. A hybrid detector may appear superior because it receives more favorable threshold tuning than a baseline; a calibrated score may be interpreted as risk even when calibration was fitted on only a few positives; and a validation false-positive budget may fail after a prevalence, host-role, or collection shift. Existing guidance identifies such evaluation pitfalls [3], but it does not by itself specify an executable method for deciding whether a detection claim remains dependable after threshold selection.

The opinions expressed here are entirely those of the authors. This research received no external funding. The authors declare no conflicts of interest.
(**Corresponding author:** Stéphane Gaël R. Ekodeck).
Author Contributions: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, and writing—original draft preparation, S.G.R.E. *and* U.J.A.N.*; conceptualization, methodology, and writing—review and editing,* S.A.E. *and* V.B.K.*; formal analysis and investigation,* S.W.M.T. *and* D.B.M.F.*; data curation and validation,* A.U.E.*; supervision, project administration, and writing—review and editing,* J.M.N. *All authors have read and approved the article.*

Stéphane Gaël R. *Ekodeck is with Department of Computer Science, LIRIMA, TEAM GRIMCAPE, University of Yaoundé I, Yaoundé, Cameroon; IRD, UMI, UMMISCO, IRD France Nord, F-93143 Bondy, France; Sorbonne Universités, Univ. Paris 06, UMI 209, UMMISCO, F-75005 Paris, France. (ORCID: 0000-0002-8094-8832, e-mail: stephane-gael.ekodeck@facsciences-uy1.cm).*

Vivien Beyala *Kamgang is with RTU-Mathematics and Computer Science, and Department of Computer Science-HTTC, The University of Bertoua, Bertoua, Cameroon; RUFCEA(UR-IFIA)-University of Dschang, Dschang, Cameroon. (ORCID: 0000-0002-3644-1866, e-mail: vivsxx@univ-bertoua.cm).*

Serge Alain *Ebélé is with Department of Computer Science, LIRIMA, TEAM GRIMCAPE, University of Yaoundé I, Yaoundé, Cameroon; IRD, UMI, UMMISCO, IRD France Nord, F-93143 Bondy, France; Sorbonne Universités, Univ. Paris 06, UMI 209, UMMISCO, F-75005 Paris, France. (ORCID: 0000-0001-7036-7068, e-mail: serge-alain.ebele@facsciences-uy1.cm).*

Ulrich Joel *Atchiengang,* Stéphane Willy *Mossebo Tcheunteu,* Damaris Belle *M. Fotso, and* Arthur Ulrich *Ewane are with University of Yaoundé I, LIRIMA, TEAM GRIMCAPE, Yaoundé, Cameroon.*

Julius Marcellin *Nkenlifack is with RUFCEA(UR-IFIA)-University of Dschang, Dschang, Cameroon; RTU-Mathematics and Computer Science, The University of Bertoua, Bertoua, Cameroon. (ORCID: 0000-0003-3322-4687, e-mail: marcellin.nkenlifack@gmail.com).*

This paper formulates the problem as *threshold-transfer evaluation*. A behavioral detector produces a score, a development procedure fixes an operating threshold, and the scientific claim concerns the fixed decision rule applied to an isolated test distribution. The central question is therefore not which architecture attains the largest isolated metric, but which ranking, threshold, calibration, and baseline conclusions remain valid when the operating point is fixed before final-test labels are available.

BETH makes this problem measurable [4]. It contains low-level endpoint events collected across multiple hosts and was designed for anomaly detection and out-of-distribution analysis. After the process-level aggregation used here, the official training and validation files contain no malicious processes, while 121 of the 198 official-test processes are malicious. An ordinary IID validation interpretation is therefore untenable: supervised development requires supplementary positives, and a threshold selected under the development distribution cannot be assumed to preserve its false-positive rate on the official test.

We address this problem with FAIR-X, a general evaluation contract for behavioral security datasets under distribution shift, and instantiate it as FAIR-BETH on BETH. Before final-test evaluation, FAIR-X fixes which records each stage may access, how labels and feature vocabularies are isolated, which threshold policies apply to every model, which non-hybrid baselines must be matched, and which uncertainty and failure disclosures accompany the point estimates. It further defines the tabular-dominance gap, score-compression gap, and threshold sensitivity so that the claim "fusion helps" becomes numerically falsifiable.

The contributions are:

- **Problem formalization:** we formulate behavioral malware evaluation as a threshold-transfer problem that separates ranking quality, validation-derived operating points, and deployable decisions under distribution shift.
- **General evaluation contract:** FAIR-X defines test-isolation, label-exclusion, development-only feature fitting, common threshold policies, baseline parity, uncertainty reporting, and claim-boundary invariants. FAIR-BETH is the BETH instantiation of this contract.
- **Quantitative decision diagnostics:** the tabular-dominance, score-compression, and threshold-sensitivity gaps compare score-only fusion, direct tabular learning, and tabular-plus-score fusion against ExtraTrees, gradient boosting, an MLP, XGBoost, and RF-500 under identical policies.
- **Threshold-transfer audit without synthetic positives:** repeated five-fold validation uses all 29 BETH development positives to fix 50 thresholds before official-test access. The narrow threshold distribution but large test-FPR expansion separates sampling stability from distributional transfer.
- **Externality and operational-boundary audits:** MalBehavD-V1 tests whether the contract can be executed on a second behavioral schema, while BETH sequence, prefix, cost, feature-stress, host-disjoint, and temporal analyses identify which conclusions do and do not support deployment.

The results support a deliberately bounded conclusion. Direct tabular forests provide the strongest thresholded BETH decisions in this study; score fusion does not demonstrate added value; and even a numerically stable development threshold does not preserve its false-positive budget under the official shift. The contribution is therefore methodological and diagnostic: FAIR-X provides a reproducible way to decide whether a behavioral malware claim is supported as a deployable decision, a ranking-only observation, or an unsupported architectural assertion. The remainder of the paper formalizes the contract, documents the data and models, reports the comparative and robustness audits, and states the resulting claim boundaries.

2 Background and Problem Setting

2.1 Behavioral Ransomware Detection

Behavioral detection monitors actions rather than static bytes. A ransomware process may create or modify many files, enumerate directories, spawn child processes, open network connections, or invoke scripting interpreters. These actions are not unique to malware: backup agents, compilers, indexers, installers, and administrative scripts can produce high-volume file and process activity. The detection problem is therefore not merely whether a suspicious event exists, but whether a process-level pattern is sufficiently different from benign behavior to justify an alert.

The common modeling approaches include unsupervised anomaly detection, supervised classification, and sequence modeling. Isolation Forest [5] recursively partitions feature space and gives higher anomaly scores to samples isolated with shorter paths. It is attractive when malicious labels are scarce but does not output calibrated probabilities. Autoencoders, including recurrent autoencoders, learn to reconstruct normal sequences and use reconstruction error as an anomaly score [6], [7]. Supervised classifiers such as Random Forests [8] can exploit labels when available but may overfit benchmark-specific artifacts unless the split design is carefully controlled.

2.2 Calibration and Operating Thresholds

Ranking metrics such as AUC-ROC and Average Precision (AP) summarize score orderings. They do not specify an operating point. In an endpoint system, a score must be converted to an action: alert, isolate, suspend, or ignore. This requires a threshold. Threshold selection is especially sensitive when validation and test prevalence differ. A default threshold of 0.5 is rarely meaningful for anomaly scores and may also be inappropriate for class-weighted classifiers.

Calibration maps raw scores into probability-like quantities. Platt scaling [9] fits a logistic regression on a calibration set; expected calibration error (ECE) bins predictions and compares confidence to empirical accuracy [10]. In extreme imbalance, however, calibration can become numerically fragile. If a calibration fold contains only a handful of positives, the fitted probabilities may cluster near the base rate, and binned calibration estimates can have high variance. We therefore treat calibration as score normalization, not as evidence that the resulting probabilities are reliable risk estimates.

2.3 Evaluation under Distribution Shift

The BETH dataset was designed for anomaly detection and out-of-distribution analysis [4]. This is important. In many benchmark papers, train and test splits are implicitly treated as exchangeable. In BETH, the official train/validation/test structure does not behave like an ordinary IID split after process aggregation. The official training and validation files contain no malicious process in our process-level representation; the official test file contains a majority of malicious processes. This creates a difficult but useful setting: the model cannot rely on an ordinary validation set to tune a malicious operating point.

We therefore distinguish between three evaluation concepts:

Definition 1 (Ranking evaluation). A ranking evaluation measures how well a model orders test processes by maliciousness, independent of any threshold.

Definition 2 (Thresholded evaluation). A thresholded evaluation measures binary decisions after the threshold has been fixed using development data.

Definition 3 (Defensible threshold policy). A threshold policy is defensible when every model being compared receives the same access to validation data and the threshold is fixed before test labels are used.

The third definition drives the rest of the paper. We report multiple threshold policies, but we do not compare a meta-classifier tuned on validation FPR against a baseline left at a default threshold.

3 Related Work and Positioning

3.1 Static Malware Learning and Its Limits

Static malware learning uses hashes, byte n-grams, imported functions, strings, headers, entropy profiles, and portable-executable metadata. Datasets such as EMBER [2] made static malware-learning research more reproducible by providing feature extraction and baseline models. Static features are attractive because they can be computed before execution and can scale to large file repositories. They are also operationally important because an endpoint product should stop known malware as early as possible.

Ransomware, however, exposes a limitation of purely static reasoning. A ransomware campaign can reuse behavior while changing packaging. A loader can alter the observed file without changing the downstream encryption logic. A static classifier may generalize across some transformations but cannot observe runtime intent directly. Behavioral detection is therefore not a replacement for static detection; it is a complementary layer that becomes important when the file-level signal is weak, hidden, or unavailable. BETH is aligned with this behavioral layer because it does not provide raw binaries or PE features.

3.2 Unsupervised Anomaly Detection

Unsupervised anomaly detection is compelling in security because positive labels are scarce and expensive. Isolation Forest is a common baseline because it is simple, fast, and robust to high-dimensional tabular inputs [5]. The general anomaly-detection literature warns, however, that an anomaly is not necessarily an attack [11]. A rare benign administrative action can be highly anomalous; a repeated malicious behavior can be statistically ordinary inside the dataset used for training.

This distinction explains why an unsupervised detector can fail under BETH distribution shift. If the benign development pool contains diverse administrative processes and the malicious test processes are internally regular, an Isolation Forest may rank benign processes as more anomalous. Such a failure is not surprising; it is a known mismatch between anomaly and maliciousness. The value of reporting it is that it prevents the paper from claiming an unsupervised branch is useful merely because it is architecturally plausible.

3.3 Sequence Models for Logs

Sequence models for logs, such as recurrent networks and transformer-style models, are often motivated by the observation that event order matters. DeepLog [7] showed that sequence prediction can be effective for log anomaly detection. A GRU autoencoder uses a similar intuition: if the model reconstructs normal event sequences well, high reconstruction error may indicate abnormal behavior. This is a reasonable hypothesis for endpoint traces because a process is not just a bag of events; the order of file, process, and network operations can encode execution semantics.

The present results show that this hypothesis is not sufficient on its own. The GRU-AE obtains AUC 0.3913 on the official test split. One explanation is that eventId sequences in BETH are low-cardinality and regular enough that benign and malicious processes share many local patterns. Another explanation is that process-level event histograms already capture most of the discriminative signal available under this aggregation. A third explanation is that the sequence model was trained only on benign processes from a development distribution that does not match the official test distribution. These explanations are not mutually exclusive.

3.4 Random Forests in Security Datasets

Random Forests [8] remain strong baselines in many tabular security datasets. They handle nonlinear interactions, tolerate mixed feature scales, provide stable performance with limited tuning, and are less data-hungry than neural models. In the context of BETH, a Random Forest can exploit interactions between event histograms and process-level scalar statistics. For example, a high event rate may be benign for a short-lived system process but suspicious when paired with a particular distribution of file and process events.

Because Random Forests are so strong on tabular data, any hybrid pipeline must compare against them carefully. A weak baseline makes a complex model appear more useful than it is. A default threshold on a class-weighted Random Forest is also insufficient: the default class prediction may be conservative under extreme imbalance. The FAIR-BETH instantiation therefore evaluates TabRF and stronger tabular alternatives under the same threshold policies as MetaRF.

3.5 Gradient Boosting and Tabular Deep Baselines

Modern tabular baselines include boosted trees such as XGBoost [12] and LightGBM [13], as well as neural tabular learners such as TabNet [14]. These methods are relevant because the BETH representation used here is primarily tabular after process aggregation. The additional audit therefore includes executable comparisons with XGBoost, ExtraTrees, scikit-learn gradient boosting, histogram gradient boosting, a multilayer perceptron, and a larger 500-tree Random Forest. LightGBM and TabNet are discussed as related tabular model families but are not required dependencies in the reproducibility package.

3.6 Security-ML Evaluation Pitfalls

FAIR-X is not intended to rediscover generic machine-learning hygiene. It is a concrete evaluation contract for a narrower failure mode in behavioral security benchmarks: a complex detector can appear useful because feature construction, threshold selection, baseline strength, or claim boundaries differ across models. This positioning follows the concerns raised by Arp et al. [3], who document recurring pitfalls in machine-learning-based security evaluation. FAIR-X turns part of that guidance into executable checks for this setting: no test-label influence on feature construction or thresholds, identical threshold policies for compared models, explicit baseline parity, uncertainty intervals, diagnostic gaps, and claim-boundary audits. Its novelty is therefore not the individual rule "do not leak test labels," but the way these rules are combined with threshold and fusion diagnostics for distribution-shifted behavioral process detection.

3.7 Threshold Transfer as an Evaluation Object

Most malware and process-behavior evaluations report either ranking metrics, such as AUC and AP, or thresholded metrics, such as F1 and recall. Both are useful, but neither alone establishes whether a fixed alerting policy is dependable. A ranking metric can be high even when the validation-derived threshold is too conservative or too aggressive under the final test distribution. Conversely, a favorable F1 value can hide the fact that the threshold was chosen with stronger label access than competing models received.

FAIR-X treats the threshold-selection rule itself as part of the evaluated system. The object being tested is not only a scorer $g(x)$, but the decision procedure (g, T), where T maps development data to an operating threshold before final-test labels are available. This distinction connects benchmark methodology to operational security: endpoint products deploy decisions, not AUC values. The threshold-transfer audit introduced here measures whether a development false-positive objective survives final-test shift, and the diagnostic gaps measure whether architectural complexity changes the deployable decision under the same threshold policy.

3.8 Calibration in Security

Calibration is often discussed as if it converts classifier scores into trustworthy probabilities. In high-stakes security, this interpretation should be used carefully. Platt scaling can improve score monotonicity and make scores comparable, but its reliability depends on the calibration data. When a calibration fold contains six positives, each positive contributes materially to the fitted logistic curve. ECE is also unstable because bins may contain very few positive examples. The present paper therefore uses calibration as a procedural normalization step and reports calibration fragility as a finding.

3.9 MITRE ATT&CK as Response Context

MITRE ATT&CK provides a useful language for tactics and techniques [15], [16]. It helps analysts connect low-level telemetry to adversary behavior and response playbooks. Yet ATT&CK technique labels are not automatically available in BETH. Mapping an event identifier such as `openat`, `execve`, or `connect` to a technique requires context. The same system call can be benign, suspicious, or malicious depending on arguments, parent process, user, host role, and temporal context.

For that reason, this paper uses ATT&CK as an analyst-facing response vocabulary rather than as a supervised feature source. This separates two different questions: whether a detector can classify malicious processes, and whether an alert can be translated into ATT&CK language through validated telemetry semantics or documented analyst rules. The former is evaluated; the latter is discussed as operational design.

4 Formal Problem Definition

Let $\mathcal{E}$ be the set of raw endpoint events. Each event e has a timestamp, a host identifier, a process identifier, a parent-process identifier, an event identifier, a textual argument field, a return value, and a label annotation. A process group is:

$$G_i = \{e \in \mathcal{E} : host(e) = h_i, processId(e) = p_i\}.$$

The process-level label is:

$$y_i = \max_{e \in G_i} \mathbf{1}\{evil(e) = 1\}.$$

The model receives a feature representation $x_i = f(G_i)$ and outputs a score $s_i = g(x_i)$, where higher scores are intended to indicate higher maliciousness.

The evaluation problem has two layers. The ranking layer measures whether:

$$s_i > s_j \quad \text{for malicious } i \text{ and benign } j.$$

The decision layer chooses a threshold τ and predicts:

$$\hat{y}_i(\tau) = \mathbf{1}\{s_i \geq \tau\}.$$

The threshold τ must be a function of development data only:

$$\tau = T(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{val}}),$$

and not a function of official test labels.

For the FPR-budget policy, let $\mathcal{V}_0$ be the benign subset of `val_strat`. We choose:

$$\tau_{\text{FPR}} = \min\{\tau : \frac{1}{|\mathcal{V}_0|} \sum_{i \in \mathcal{V}_0} \mathbf{1}\{s_i \geq \tau\} \leq \alpha\},$$

with $\alpha = 0.05$. For the validation-F1 policy, let $\mathcal{V}$ be the complete stratified validation fold. We choose:

$$\tau_{F1} = \arg\max_{\tau \in \mathcal{T}} F1(\mathcal{V}, \tau),$$

where $\mathcal{T}$ is the set of observed validation scores. The FPR policy encodes an operational constraint; the F1 policy encodes a data-driven trade-off that is less stable when positives are scarce.

This formalization clarifies the main fairness requirement. If two models are compared, the mapping T from validation scores to threshold must be the same type of mapping. Otherwise, the comparison confounds the classifier with the threshold policy.

5 FAIR-X Evaluation Contract and FAIR-BETH Instantiation

FAIR-X is the evaluation methodology proposed in this paper. It is not a new feature extractor or classifier. It is a contract for deciding whether a behavioral security model supports a deployable decision claim under distribution shift. The contract specifies what information a model may use, how thresholds are selected, how baselines are compared, and which diagnostic quantities must be reported before claiming that a detector architecture improves over a simpler baseline. FAIR-BETH is the concrete instantiation of FAIR-X for process-level BETH evaluation.

5.1 General FAIR-X Evaluation Contract

FAIR-X applies to behavioral security datasets whose development and test distributions may differ. Let $\mathcal{D}_{dev}$ be all data available before final testing, $\mathcal{D}_{test}$ be the isolated final test set, $\mathcal{F}$ be a feature-construction operator, $\mathcal{T}$ be a threshold-selection operator, and $\mathcal{M} = \{M_1, \ldots, M_k\}$ be the candidate models. A FAIR-X experiment is the tuple

$$\mathcal{E} = (\mathcal{D}_{dev}, \mathcal{D}_{test}, \mathcal{F}, \mathcal{M}, \mathcal{T}, \mathcal{A}),$$

where $\mathcal{A}$ is a declared audit set. The tuple is valid only if the following invariants hold before test evaluation:

$$\mathcal{F} \perp Y_{test}, \qquad \mathcal{T} \perp Y_{test}, \qquad \forall M_i, M_j \in \mathcal{M} : \mathcal{T}_i = \mathcal{T}_j$$

for every model comparison used to support a scientific claim. The first two invariants exclude test-label leakage through feature construction and threshold selection. The third invariant enforces threshold fairness: a hybrid model cannot be compared against a baseline that receives a weaker operating-point policy.

The audit set $\mathcal{A}$ must include at least one baseline-parity audit, one threshold-sensitivity audit, and one uncertainty audit. For deployment-oriented claims, $\mathcal{A}$ should also include prefix latency, cost sensitivity, and split robustness. The BETH instantiation uses process aggregation, direct tabular baselines, MetaRF as the hybrid candidate, validation-FPR and validation-F1 threshold policies, bootstrap intervals, diagnostic gaps, and the limit-lifting audits in Section 12. Figure 1 summarizes the resulting contract.

Definition 4 (FAIR-X improvement claim). A model M_a has a defensible improvement claim over M_b under policy π only if both models use the same admissible training information, the same threshold policy π, no test labels in feature construction or threshold selection, and the reported uncertainty does not contradict the claimed effect direction.

Principle 1 (Fusion burden of proof). Under a FAIR-X contract, a hybrid detector does not earn its added complexity unless its test-set utility exceeds the strongest admissible non-hybrid baseline under the same threshold policy and audit set.

The principle is operational rather than asymptotic. It defines the burden of proof used in this paper: MetaRF is not rejected because fusion is impossible; it fails because, under the same threshold policy and data access, it does not improve over the strongest admissible tabular baseline on the evaluated BETH configuration.

Algorithm 1 FAIR-X evaluation protocol instantiated as FAIR-BETH

Require: Official BETH train/validation/test files, supplementary development files, target FPR budget α
Ensure: Test metrics, confidence intervals, and diagnostic gaps

1: Aggregate events by $(hostName, processId)$ and define process labels from `evil`
2: Keep annotation fields out of predictors and reserve ATT&CK mappings for response context unless validated
3: Build an enriched development pool without using official test labels
4: Split development data into train, calibration, and stratified validation folds
5: Fit feature vocabularies and preprocessing transforms on development data only
6: Train raw detectors, supervised tabular baselines, score-only fusion, and meta-fusion
7: Calibrate raw detector scores on the calibration fold
8: Select τ_{FPR} and τ_{F1} on the validation fold for every model
9: Evaluate all fixed models and thresholds once on the official test split
10: Compute ranking metrics, thresholded metrics, confusion matrices, bootstrap intervals, and diagnostic gaps

5.2 Protocol Requirements

The protocol has six requirements. First, the official test split is isolated from model fitting, calibration, threshold selection, feature-vocabulary construction, and model selection. Second, annotation columns such as `sus` and `evil` are used only to construct labels and never as predictors. Third, ATT&CK-related annotations are used as predictive variables only when the dataset provides validated technique labels or documented mapping rules. Fourth, all models are evaluated under the same threshold policies. Fifth, every hybrid model is compared against the strongest reasonable non-hybrid baseline using the same data access and threshold policy. Sixth, uncertainty and failure modes are reported alongside point estimates. Algorithm 1 gives the resulting execution order.

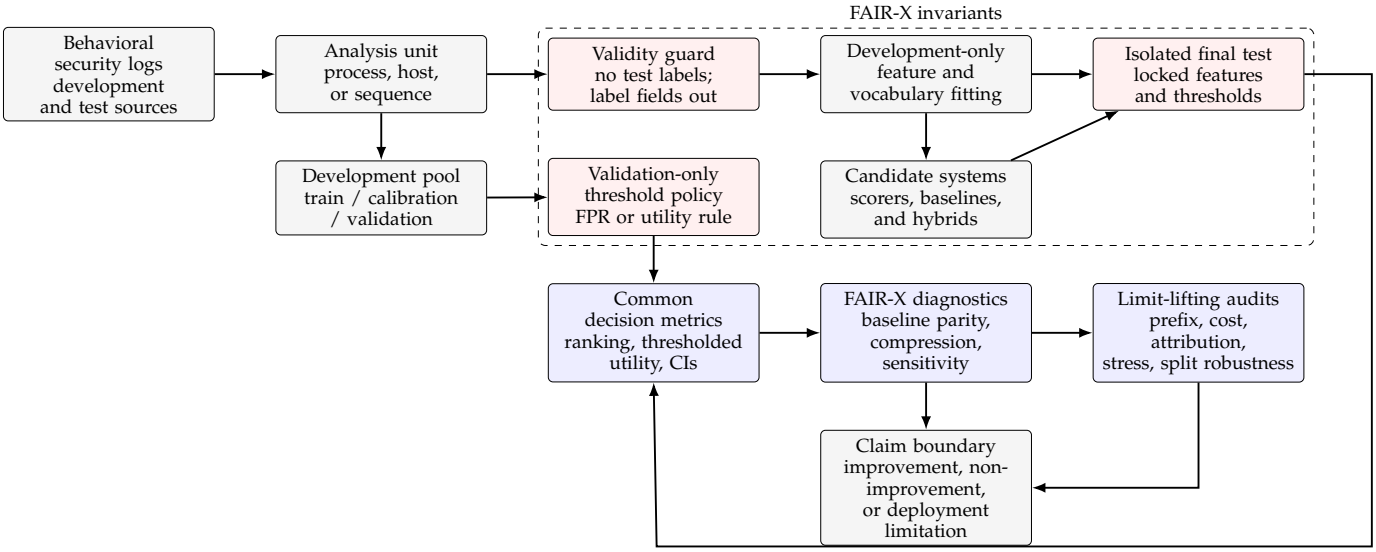


Fig. 1. General FAIR-X threshold-transfer evaluation contract. Red blocks are non-negotiable validity guards; blue blocks are audits that determine whether a performance claim, negative result, or deployment boundary is defensible. FAIR-BETH instantiates this contract for process-level BETH evaluation.

5.3 Diagnostic Gaps

The novelty of FAIR-X is partly procedural and partly diagnostic. The diagnostic quantities below are designed to expose whether a complex detector actually adds information beyond a strong baseline under the same validation access and threshold policy.

Let M_{base} denote the strongest admissible non-hybrid tabular baseline, M_{score} the score-only fusion model, M_{meta} the meta-fusion model, and π a fixed threshold policy. The *tabular-dominance gap* is:

$$\text{TDG}_\pi = F1(M_{\text{base}}, \pi) - F1(M_{\text{meta}}, \pi).$$

A positive value means the hybrid meta-model does not improve over the tabular baseline under the same threshold policy. The *score-compression gap* is:

$$\text{SCG}_\pi = F1(M_{\text{base}}, \pi) - F1(M_{\text{score}}, \pi).$$

A large positive value means that detector scores compress away useful behavioral information. Finally, the *threshold sensitivity* of model M is:

$$\text{TS}_M = |F1(M, \tau_{\text{FPR}}) - F1(M, \tau_{F1})|.$$

This measures how much the reported F1 changes when the validation objective changes. A large value warns that claims based on one threshold policy may not be stable.

These gaps deliberately use F1 because the main comparison concerns thresholded operational decisions. They do not replace AUC, AP, or confidence intervals. Rather, they summarize the practical question at the center of the experiment: after giving every model the same validation access, does fusion still help?

5.4 Scope of the Protocol

FAIR-X is intentionally scoped. It applies to behavioral security experiments where development and final-test distributions may differ and where threshold selection is part of the scientific claim. FAIR-BETH is the process-level BETH instantiation of that scope. The contract does not by itself solve family attribution, direct cross-schema model transfer, malware-level evasion, or response safety. The additional audits quantify feature attribution, prefix latency, operational cost sensitivity, and split robustness, but those measurements remain evidence about the evaluated datasets rather than deployment certification. The purpose of FAIR-X is to prevent an avoidable methodological error: claiming novelty from a detector architecture when the measured gain comes from unequal thresholding, label leakage, weak baselines, or feature sources whose supervision or semantics are not available at deployment time.

6 Dataset and Data Engineering

6.1 BETH Files and Process-Level Labels

BETH consists of low-level events with fields such as `timestamp`, `processId`, `parentProcessId`, `eventId`, `eventName`, `argsNum`, `returnValue`, `args`, `hostName`, `sus`, and `evil`. The official files and supplementary 2021may files do not have identical schemas: the official files include fields such as `threadId`, `mountNamespace`, and `stackAddresses`, while the supplementary files omit some of these fields. To avoid schema-dependent artifacts, we use only the fields present across the files needed for the experiment.

We aggregate events by the pair (`hostName`, `processId`). A process is labeled malicious if any event in the group has `evil` $= 1$. The label fields are used only for this group label. They are excluded from all feature vectors. This choice is essential because `sus` and `evil` are annotations, not observables available to a deployed detector.

Table 1 summarizes the event-level and process-level inventory used by the pipeline. The official training and validation files contain no malicious process after aggregation. The official test file contains 121 malicious processes out of 198. The supplementary 2021may files provide the

positive development examples needed for calibration and validation, but they do not remove the distribution shift between development and official test.

6.2 Development and Test Protocol

Because the official training and validation splits are monoclasse at process level, they cannot be used alone to train and calibrate supervised process-level detectors. We therefore construct an enriched development pool from the official training file plus the non-DNS supplementary 2021may files. The official test file remains isolated and is used only once for final evaluation.

The enriched development pool is split into:

- **train** (65%): model fitting and detector training;
- **cal** (20%): Platt scaling and calibration diagnostics;
- **val_strat** (15%): threshold selection.

The split is stratified by process-level label. Table 2 gives the resulting process counts. The positive counts are small: 19 in train, 6 in calibration, and 4 in stratified validation. This is not ideal, but it is the only way to define a threshold without touching the official test labels. We explicitly report this as a limitation and use bootstrap intervals to avoid overclaiming precision.

6.3 Feature Construction

For each process, we build a behavioral vector from scalar statistics and normalized event histograms. The final tabular vector has 61 dimensions: 13 scalar process statistics and 48 eventId histogram dimensions learned from the enriched development vocabulary. The scalar features are:

- number of events;
- number of unique event identifiers;
- Shannon entropy of the event identifier distribution;
- mean, standard deviation, and maximum of `argsNum`;
- mean and standard deviation of `returnValue`;
- process duration;
- event rate;
- number of unique argument strings;
- mean argument-string length;
- number of unique parent process identifiers.

The histogram part of the feature vector is normalized by the number of events in the process. This prevents long processes from dominating solely because they contain more events. Process length remains available through n_{events} and event rate.

6.4 Sequence Representation

For the GRU autoencoder, each process is represented as a sequence of event identifiers. Event identifiers are mapped to integer tokens. Token 0 is padding, and token 1 is reserved for unknown identifiers. The sequence length is set to the 95th percentile of training sequence lengths, which is 1 152 events. Shorter sequences are padded. Longer sequences are uniformly sampled with evenly spaced indices. Uniform sampling avoids the strong bias introduced by truncating only the beginning or only the end of long traces.

6.5 Why ATT&CK Is Used as Response Context

MITRE ATT&CK is valuable for analyst interpretation and response planning [15], [16]. However, BETH event identifiers do not include validated ATT&CK technique labels or documented EDR mapping rules. Technique assignment depends on arguments, parent process, user, file path, network context, host role, and event sequence. The experiment therefore does not treat ATT&CK technique IDs as supervised predictors. This is not a rejection of ATT&CK; it is a boundary on what the present data can support. The supplementary response notes use ATT&CK as an analyst taxonomy rather than as an evaluated predictive feature.

7 Models

7.1 Isolation Forest with Platt Scaling

The first detector is an Isolation Forest [5] with 200 trees. The contamination parameter is set to the empirical malicious rate in the training fold. Input features are standardized using parameters learned on the training fold. The raw anomaly score is the negative of the scikit-learn `decision_function`; larger values are intended to indicate greater abnormality. Because the score is not a probability, we fit Platt scaling on the calibration fold.

This detector is deliberately simple. It represents the hypothesis that malicious processes are isolated in process-level feature space without requiring many labels. If the detector fails, the failure is informative because it indicates that the malicious test processes are not merely outliers relative to the development processes.

7.2 GRU Autoencoder with Platt Scaling

The second detector is a GRU autoencoder trained only on benign training sequences. It has an embedding dimension of 32, hidden dimension of 64, one recurrent layer, and 42 482 trainable parameters. The decoder reconstructs the input event sequence, and the anomaly score is the mean negative log-likelihood per non-padding token. We use Adam with learning rate 10^{-3}, batch size 512, and early stopping against the official validation reconstruction loss. The official validation split contains only benign processes, so it is suitable for monitoring benign reconstruction but not for selecting a malicious operating threshold.

The autoencoder tests whether sequential order adds useful information beyond event histograms. A recurrent model should in principle capture local event ordering, repeated motifs, and unusual process trajectories. In practice, the results show that this sequence signal does not dominate the simpler tabular representation on BETH.

7.3 Score-Only Random Forest

The score-only Random Forest, denoted ScoreRF, uses only calibrated detector scores and their simple average as features. Its input is therefore:

$$X_{\text{score}} = [p_{\text{IF}}, p_{\text{GRU}}, (p_{\text{IF}} + p_{\text{GRU}})/2].$$

This model isolates the contribution of detector fusion. If ScoreRF performs well, score-level fusion is sufficient. If it performs poorly while direct tabular baselines perform well, the raw behavioral features contain information lost by the detector scores.

TABLE 1
BETH file inventory after process aggregation. DNS files are excluded.

File	Events	Evil events	Processes	Evil processes	Unique eventIds
labelled_training_data.csv	763 144	0	1 179	0	32
labelled_validation_data.csv	188 967	0	244	0	29
labelled_testing_data.csv	188 967	158 432	198	121	46
labelled_2021may-ubuntu.csv	199 222	0	298	0	17
labelled_2021may-ip-10-100-1-4.csv	485 241	4 394	809	6	45
labelled_2021may-ip-10-100-1-26.csv	378 424	0	449	0	34
labelled_2021may-ip-10-100-1-95.csv	479 433	0	1 124	0	38
labelled_2021may-ip-10-100-1-105.csv	409 931	5 611	599	23	47
labelled_2021may-ip-10-100-1-186.csv	713 867	0	1 505	0	33

TABLE 2
Process-level splits used for model development and final evaluation.

Split	Total	Malicious	Rate
Train	3 782	19	0.50%
Calibration	1 163	6	0.52%
Stratified validation	874	4	0.46%
Official validation	244	0	0.00%
Official test	198	121	61.11%

7.4 Tabular Random Forest

The tabular Random Forest, denoted TabRF, is a supervised classifier trained on the 61-dimensional process feature vector. It uses 200 trees, balanced class weights, no fixed maximum depth, and minimum leaf size 2. Inputs are standardized before fitting. Although Random Forests do not require feature scaling for split decisions, the same scaling interface is maintained across the codebase for reproducibility and comparability.

TabRF is the main ablation baseline, and RF-500 is added as a larger-tree tabular stress baseline. These are not weak baselines; they directly test whether the engineered process features are already sufficient. A hybrid model must beat the strongest admissible tabular baseline under the same threshold policy to justify added complexity.

7.5 Meta Random Forest

The meta Random Forest, denoted MetaRF, concatenates score-level features with the raw tabular vector:

$$X_{\text{meta}} = [p_{\text{IF}}, p_{\text{GRU}}, p_{\text{avg}}, X_{\text{tab}}].$$

It uses 300 trees, balanced class weights, and minimum leaf size 2. It is trained on the union of train and calibration folds after Platt scaling has been fit. The stratified validation fold is not used for fitting; it is reserved for threshold selection.

MetaRF is intentionally evaluated as an ablation, not as the presumed best model. The central question is whether adding calibrated detector scores to raw features improves over direct tabular learning. The answer in Section 10 is no.

8 Thresholding and Metrics

8.1 Threshold Policies

We evaluate two threshold policies for every model:

- **Validation-FPR budget**: choose the most permissive validation threshold (the lowest threshold value) satisfying $\text{FPR} \leq 0.05$ on `val_strat`.
- **Validation-max-F1**: choose the validation threshold that maximizes F1 on `val_strat`.

Both policies are imperfect because `val_strat` contains only four malicious processes. The FPR-budget policy is more operationally meaningful: it reflects the usual security requirement that false positives be limited before deployment. The max-F1 policy is reported as a sensitivity analysis and is not used as the main claim.

The critical methodological point is that both policies are applied to every model. This prevents the following common but invalid comparison: a meta-classifier is evaluated at a tuned threshold while the baseline is evaluated at 0.5. Such a comparison confounds model quality with threshold policy.

8.2 Ranking and Thresholded Metrics

We report AUC-ROC and AP as ranking metrics. AP is particularly relevant under class imbalance because it summarizes precision-recall behavior and is sensitive to the positive class prevalence [17]. For thresholded metrics we report precision, recall, F1, and the confusion matrix.

Let TP, FP, TN, and FN denote true positives, false positives, true negatives, and false negatives. The reported binary metrics are:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

When the denominator is zero, the metric is set to zero.

8.3 Bootstrap Confidence Intervals

The official test set contains only 198 processes. Point estimates can therefore be unstable. We compute nonparametric bootstrap confidence intervals by resampling the test processes with replacement 1 000 times [18]. For each bootstrap sample containing both classes, we recompute AUC, AP, precision, recall, and F1 at the fixed threshold. The 2.5th and 97.5th percentiles define the 95% interval.

The bootstrap is not a substitute for cross-dataset validation. It quantifies sampling uncertainty conditional on the test set. We use it to avoid overstating small differences

such as AUC 0.736 versus 0.711. The separate MalBehavD-V1 experiment in Section 11 tests whether the protocol itself transfers to a second behavioral schema; it is not used to claim direct BETH-model transfer.

9 Implementation and Reproducibility

All experiments run on an HP ZBook 15 G5 with Intel Core i7-8750H CPU, 12 logical cores, 15.64 GB RAM, no GPU, Windows 11, Python 3.13.5, PyTorch 2.8.0+cpu, scikit-learn 1.7.2, pandas 2.3.1, NumPy 2.2.6, matplotlib 3.10.5, seaborn 0.13.2, tqdm 4.67.1, and psutil 7.0.0. The full CPU run, including preprocessing, GRU training, calibration, ablations, evaluation, and figure generation, completed in approximately 21 minutes on the test machine.

The code fixes the random seed at 42 for Python, NumPy, and PyTorch. The preprocessing stage writes five pickled datasets: train, calibration, stratified validation, official validation, and official test. The model stage writes detector outputs and fitted models. The evaluation stage writes a CSV table containing all metrics and confidence intervals, plus ROC, precision-recall, and confusion-matrix figures. This separation is useful because it prevents accidental reuse of test labels during model fitting and makes it possible to inspect each artifact independently.

10 Results

10.1 Full Quantitative Results

Table 3 gives the full test-set results for the principal FAIR-X/FAIR-BETH ablation set. Under the validation-FPR budget, TabRF obtains AUC 0.7360, AP 0.8270, precision 0.7500, recall 0.6942, and F1 0.7210. Its confusion matrix is TN=49, FP=28, FN=37, TP=84. MetaRF has lower AUC, AP, recall, and F1 under the same policy. ScoreRF improves over the unsupervised detectors but remains much weaker than TabRF.

The result contradicts the tempting narrative that a hybrid detector automatically improves over a tabular baseline. On this benchmark, with this split, and under common thresholding, detector-score fusion does not improve over direct tabular learning. This is the principal empirical finding.

10.2 Comparison with Stronger Tabular Baselines

Table 4 adds stronger tabular baselines trained on the same development information and evaluated under the same validation-FPR threshold policy. TabRF obtains the highest F1 point estimate, 0.7210; RF-500 is close at 0.7043 and its interval overlaps TabRF's interval. The two confusion matrices share the same false-positive count, but RF-500 misses three additional malicious processes. ExtraTrees has the best AUC and AP but selects a conservative operating point with lower recall. XGBoost, histogram gradient boosting, standard gradient boosting, and the MLP do not improve the thresholded F1 under this validation policy.

These comparisons sharpen the claim boundary. FAIR-X should not be read as evidence that one Random Forest configuration is uniquely optimal. It shows that a family of direct tabular learners is competitive and that score-level fusion does not earn its complexity under the fixed protocol. The comparison with the original BETH paper is necessarily qualitative: Highnam et al. report event-level OoD AUROC on an initial preprocessed subset for robust covariance (0.519), one-class SVM (0.605), iForest (0.850), and VAE+DoSE-SVM (0.698) [4]. Those results are not directly commensurable with the present process-level F1/AP/AUC tables because this paper changes the unit of analysis from events to aggregated processes, introduces supplementary development positives for supervised threshold selection, and evaluates fixed process-level decisions on the official test file. The MalBehavD-V1 comparison is also not a state-of-the-art claim: API-MalDetect and related deep API-call models target the same public dataset family [19], while this paper uses MalBehavD-V1 to test whether the FAIR-X protocol can be executed on a second behavioral schema.

10.3 FAIR-X Diagnostic Results

Table 5 reports the FAIR-X diagnostic gaps under the final test evaluation. Under the validation-FPR policy, the tabular-dominance gap is $+0.0454$: TabRF exceeds MetaRF in F1 despite using a simpler non-hybrid feature set. The score-compression gap is $+0.2479$: reducing behavior to calibrated detector scores loses substantial information relative to direct tabular learning. Threshold sensitivity is also large. TabRF changes by 0.1766 F1 points between the validation-FPR and validation-max-F1 policies, while MetaRF changes by 0.2228. These values explain why a single thresholded number is not enough to support a strong architectural claim.

The diagnostic table is central to the contribution. It gives readers a reusable audit method: compute whether fusion helps, quantify how much information is lost by score compression, and measure how sensitive conclusions are to the threshold policy. This shifts the contribution from a model-performance claim to a protocol-and-diagnostics claim.

10.4 Confusion Matrices at the Operational Threshold

Table 6 compares the three supervised ablations at the validation-FPR threshold. ScoreRF misses most malicious processes. MetaRF improves substantially but still misses 45 of 121 malicious processes. TabRF detects 84 malicious processes and misses 37, at the cost of 28 false positives among 77 benign test processes. The resulting test FPR is 36.4%, much larger than the validation budget. This discrepancy is another manifestation of distribution shift.

Figures 2 and 3 show the ranking curves, and Figure 4 visualizes the operational confusion matrices. TabRF and MetaRF have stronger ranking metrics than the unsupervised detectors. The gap between TabRF and MetaRF is not large, and their confidence intervals overlap. The defensible conclusion is therefore not that TabRF is universally superior; it is that the additional score-fusion features do not provide evidence of improvement on this test set.

10.5 Effect of Threshold Policy

The validation-max-F1 policy behaves differently from the validation-FPR policy. For TabRF, max-F1 on validation

TABLE 3
Complete test-set results with common threshold policies. Confidence intervals are 95% nonparametric bootstrap intervals.

Model	Threshold policy	AUC	AP	Precision	Recall	F1	F1 95% CI
IF-Platt	val-FPR budget	0.3496	0.5195	0.3500	0.0579	0.0993	[0.0303, 0.1727]
IF-Platt	val-max-F1	0.3496	0.5195	0.4590	0.2314	0.3077	[0.2128, 0.3895]
GRUAE-Platt	val-FPR budget	0.3913	0.5347	0.6250	0.0413	0.0775	[0.0163, 0.1475]
GRUAE-Platt	val-max-F1	0.3913	0.5347	0.5638	0.4380	0.4930	[0.4095, 0.5702]
SimpleAvg	val-FPR budget	0.3402	0.5089	0.2778	0.0413	0.0719	[0.0150, 0.1379]
SimpleAvg	val-max-F1	0.3402	0.5089	0.5060	0.3471	0.4118	[0.3179, 0.5022]
ScoreRF	val-FPR budget	0.5960	0.6846	0.6769	0.3636	0.4731	[0.3825, 0.5650]
ScoreRF	val-max-F1	0.5960	0.6846	0.6852	0.3058	0.4229	[0.3250, 0.5165]
TabRF	**val-FPR budget**	**0.7360**	**0.8270**	**0.7500**	**0.6942**	**0.7210**	**[0.6484, 0.7849]**
TabRF	val-max-F1	0.7360	0.8270	0.9583	0.3802	0.5444	[0.4512, 0.6304]
MetaRF	val-FPR budget	0.7113	0.8105	0.7308	0.6281	0.6756	[0.5990, 0.7456]
MetaRF	val-max-F1	0.7113	0.8105	0.9474	0.2975	0.4528	[0.3544, 0.5476]

TABLE 4
Additional tabular baselines on the official BETH test split under the common validation-FPR threshold policy. Confidence intervals are 95% nonparametric bootstrap intervals.

Model	AUC	AP	Precision	Recall	F1	F1 95% CI	FP	FN
TabRF	0.7360	0.8270	0.7500	0.6942	0.7210	[0.6484, 0.7849]	28	37
RF-500	0.7490	0.8352	0.7431	0.6694	0.7043	[0.6372, 0.7687]	28	40
MetaRF	0.7113	0.8105	0.7308	0.6281	0.6756	[0.5990, 0.7456]	28	45
ExtraTrees	0.7973	0.8572	0.9649	0.4545	0.6180	[0.5294, 0.7005]	2	66
HistGradientBoosting	0.7106	0.8090	0.7033	0.5289	0.6038	[0.5237, 0.6791]	27	57
XGBoost	0.6717	0.7841	0.7176	0.5041	0.5922	[0.5052, 0.6667]	24	60
MLP	0.7865	0.8339	0.9231	0.3967	0.5549	[0.4568, 0.6409]	4	73
GradientBoosting	0.6153	0.7552	0.6571	0.3802	0.4817	[0.3889, 0.5688]	24	75
ScoreRF	0.5960	0.6846	0.6769	0.3636	0.4731	[0.3825, 0.5650]	21	77
LogReg	0.7698	0.8024	0.7059	0.0992	0.1739	[0.0916, 0.2571]	5	109

TABLE 5
FAIR-X diagnostic gaps computed from the fixed official test evaluation. Positive TDG means the tabular baseline has higher F1 than meta-fusion under the same policy.

Diagnostic	Formula	Value	Interpretation
Tabular-dominance gap	$F1(\text{TabRF}) - F1(\text{MetaRF})$ under val-FPR	$+0.0454$	Fusion does not improve over the strongest tabular baseline
Score-compression gap	$F1(\text{TabRF}) - F1(\text{ScoreRF})$ under val-FPR	$+0.2479$	Detector scores discard useful behavioral detail
TabRF threshold sensitivity	$\lvert F1_{\text{valFPR}} - F1_{\text{valF1}}\rvert$	0.1766	TabRF is materially threshold-dependent
RF-500 threshold sensitivity	$\lvert F1_{\text{valFPR}} - F1_{\text{valF1}}\rvert$	0.1774	RF-500 is also materially threshold-dependent
MetaRF threshold sensitivity	$\lvert F1_{\text{valFPR}} - F1_{\text{valF1}}\rvert$	0.2228	MetaRF is even more threshold-dependent

TABLE 6
Confusion matrices for supervised models under the validation-FPR budget policy.

Model	TN	FP	FN	TP
ScoreRF	56	21	77	44
TabRF	49	28	37	84
MetaRF	49	28	45	76

selects a much higher threshold, yielding precision 0.9583 but recall 0.3802 and F1 0.5444 on test. For MetaRF, the corresponding threshold yields precision 0.9474 but recall 0.2975 and F1 0.4528. Thus, optimizing F1 on a validation set with four positives does not generalize to the official test split.

This is a practical lesson: a validation set can be stratified but still too small to support stable threshold optimization. A threshold policy based on a false-positive budget is easier to justify operationally, but it also fails to preserve the target FPR under severe distribution shift. In the present experiment, the threshold is not wrong in a procedural sense; it is wrong because the validation distribution is not representative of the test distribution.

10.6 Repeated Cross-Fitted Threshold Audit

To reduce dependence on the four-positive `val_strat` fold, we perform a second, predeclared threshold analysis using all 5 819 development processes (29 malicious, 5 790 benign). A 5-fold stratified split is repeated ten times. In each of the 50 folds, an RF-500 model is fitted on four

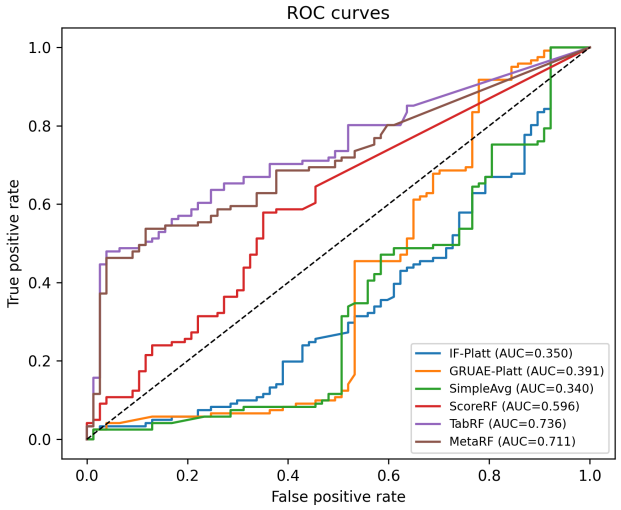


Fig. 2. ROC curves on the official BETH test split. The tabular and meta Random Forests have stronger ranking behavior than the raw detector scores.

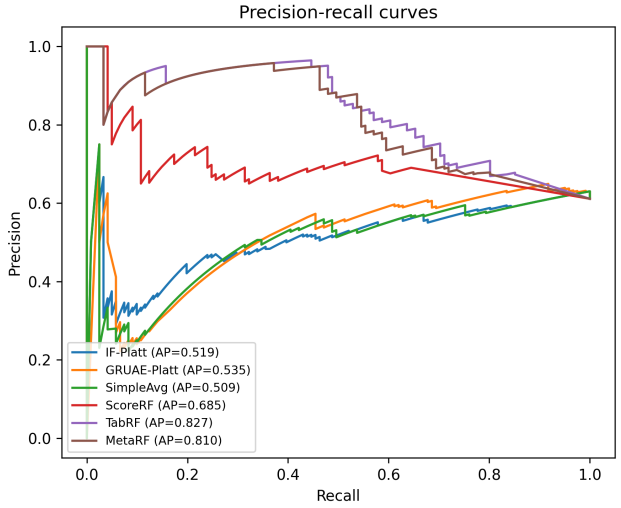


Fig. 3. Precision-recall curves on the official BETH test split. AP is reported in the legend of the generated figure.

folds and its 5%-FPR threshold is selected on the held-out fold, which contains five or six positives. No synthetic process is generated, and the official test scores and labels are unavailable throughout this procedure. The median of the 50 thresholds is then locked; a final RF-500 model is fitted on the complete development pool and evaluated once on the official test. Table 7 summarizes this audit.

The threshold distribution is narrow, and propagating every preselected threshold to the test yields F1 between 0.6964 and 0.7124. Thus, the original four-positive fold is not solely responsible for the broad conclusion that direct tabular learning is useful. However, the development FPR constraint still fails to transfer: the locked median threshold produces 25 false positives among 77 benign test processes. Repeated validation reduces threshold sampling dependence; it cannot repair the underlying population shift or increase the 29 independent positive development processes.

TABLE 7
Repeated cross-fitted threshold audit. Thresholds are fixed before official-test evaluation.

Quantity	Result
Development processes (positive)	5 819 (29)
Repeated folds	$5 \times 10 = 50$
Positives per held-out fold	5–6
Threshold median (IQR)	0.006 (0.004–0.006)
Locked-test precision	0.7573
Locked-test recall	0.6446
Locked-test F1	0.6964
Locked-test FPR	0.3247

TABLE 8
Target-copy-free next-event sequence ablation on BETH.

Model	Params.	AUC	AP	F1
GRU-64×1	23 666	0.4160	0.5904	0.1471
GRU-128×2	183 218	0.4911	0.6386	0.1333
LSTM-128×2	241 074	0.4722	0.6305	0.1353

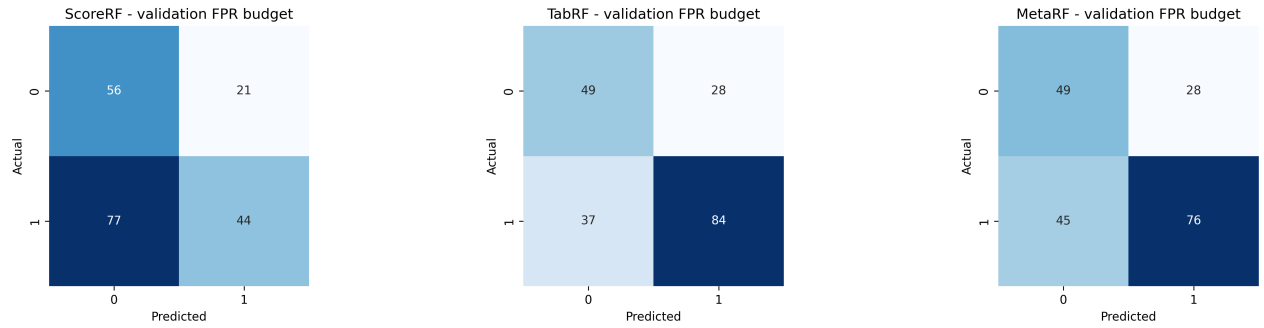
10.7 Target-Copy-Free Sequence Ablation

The reconstruction branch allows the decoder to observe the token sequence it reconstructs. We therefore add a stricter causal objective: each recurrent model receives tokens $1, \ldots, t$ and predicts token $t + 1$. Models are trained only on 3 763 benign training processes, use the first 512 tokens, and use the all-benign official validation file to set an empirical 5%-FPR anomaly threshold. Score orientation is fixed before test evaluation: higher next-event negative log-likelihood is more anomalous. Table 8 varies recurrent cell, depth, and capacity.

Increasing recurrent capacity and changing the cell do not recover a useful operating point: test recall remains between 0.0744 and 0.0826. This rules out the specific explanation that the initial result arose only from a 42 482-parameter GRU. It does not establish that every sequence model is unsuitable; supervised sequence learning, transformers, process trees, alternative token semantics, and longer training searches remain outside this experiment.

10.8 Calibration Audit

Table 9 reports expected calibration error and Brier score on the official test split for the supervised tabular models; the supplementary material provides the reliability diagrams. ECE values near 0.50 indicate near-null calibration: calibrated scores carry no reliable probability interpretation on the BETH test distribution. TabRF has ECE 0.4984 and Brier score 0.4506, while MetaRF has ECE 0.5007 and Brier score 0.4621. Platt scaling normalizes score ranges but does not produce meaningful posterior estimates under this prevalence mismatch. Logistic regression has the lowest ECE among the audited models (0.2905), but its validation-FPR F1 is only 0.1739. Calibration quality and operational detection quality therefore diverge, and the final claims treat model outputs as scores for thresholding rather than reliable posterior probabilities.



(a) ScoreRF (b) TabRF (c) MetaRF

Fig. 4. Confusion matrices under the validation-FPR budget policy. TabRF detects the most malicious processes while maintaining the same false-positive count as MetaRF.

TABLE 9
Calibration audit on the official BETH test split. Lower ECE and Brier scores indicate better calibration, but these values should be read together with thresholded detection performance.

Model	ECE	Brier
LogReg	0.2905	0.2734
XGBoost	0.3876	0.3725
ExtraTrees	0.3932	0.3498
RF-500	0.4994	0.4481
TabRF	0.4984	0.4506
MetaRF	0.5007	0.4621
GradientBoosting	0.5029	0.4980
HistGradientBoosting	0.5177	0.5124
ScoreRF	0.5378	0.5243
MLP	0.5913	0.5782

10.9 Ablation Interpretation

The ablation structure isolates three information levels:

- calibrated anomaly scores only (ScoreRF);
- raw aggregated behavioral features only (TabRF);
- raw features plus calibrated scores (MetaRF).

ScoreRF has F1 0.4731 under the main policy. This means the two detector scores contain some signal, but not enough. TabRF and RF-500 reach F1 0.7210 and 0.7043, indicating that the event histogram and scalar process features are the dominant source of predictive information. MetaRF has F1 0.6756, showing that adding score features does not improve the Random Forest. The likely reason is redundancy: the Isolation Forest and GRU-AE scores are derived from the same event and sequence information already available to the supervised tree ensemble, but in a compressed form that loses discriminative detail.

10.10 Why the Individual Detector Scores Invert

The Isolation Forest has AUC 0.3496 and the GRU-AE has AUC 0.3913. AUC below 0.5 means the score ordering is partially inverted: many malicious test processes receive lower anomaly scores than benign test processes. This can happen when the benign development distribution contains long, diverse, or noisy processes that look more anomalous than the malicious processes in the official test split. It can also occur if malicious test traces are internally consistent and therefore reconstructed well by a sequence model trained on benign event syntax.

The target-copy-free ablation in Section 10.7 removes one architectural shortcut and still obtains AUC below 0.5 across GRU and LSTM variants. The inversion is therefore not confined to the original decoder design or its parameter count. It remains conditional on the tested next-event and reconstruction objectives, token representation, training data, and BETH split; it does not justify a general claim about all sequence models.

11 External Validation on MalBehavD-V1

To test whether FAIR-X is useful beyond BETH, we apply the same evaluation principles to MalBehavD-V1 [19], [20], an independent dynamic Windows malware dataset. MalBehavD-V1 contains API-call sequences extracted from benign and malware executables executed in a Cuckoo sandbox. The public CSV used here contains 2 570 samples: 1 285 benign files and 1 285 malware files. Sequence length ranges from 2 to 175 API calls, with median length 37 and 291 unique API names.

The external experiment is not a direct transfer of the BETH-trained TabRF model. Direct transfer would be invalid because BETH contains endpoint event records, while MalBehavD-V1 contains ordered Windows API-call sequences. Instead, the purpose is protocol validation: the same FAIR-X rules are applied to an independent behavioral dataset. Labels are excluded from predictors; feature vocabularies are fit only on training data; thresholds are selected only on validation data; and oracle test thresholds are reported only as non-deployable upper bounds.

We represent each API-call sequence by scalar sequence statistics, API-family rates, and normalized API histograms using the 128 most frequent training APIs. The Random Forest uses 300 trees, balanced class weights, and minimum leaf size 2. The fixed split allocates 60% of the data to training, 15% to calibration, 10% to validation, and 15% to testing. We also run 30 repeated stratified splits to estimate split sensitivity.

Table 10 shows that the FAIR-X contract is not tied to BETH-specific artifacts. Under a validation-FPR threshold, the external Random Forest reaches F1 0.9512 with 11 false positives and 8 false negatives on the held-out test

TABLE 10
External validation on MalBehavD-V1 using full API-call sequences. The oracle threshold is shown only as a non-deployable upper bound.

Threshold policy	AUC	AP	Precision	Recall	F1	F1 95% CI	FP	FN
Validation-FPR budget	0.9902	0.9922	0.9439	0.9585	0.9512	[0.9291, 0.9720]	11	8
Validation-max-F1	0.9902	0.9922	0.9735	0.9534	0.9634	[0.9430, 0.9809]	5	9
Oracle test max-F1	0.9902	0.9922	0.9737	0.9585	0.9661	[0.9468, 0.9834]	5	8

TABLE 11
Prefix-length evaluation on MalBehavD-V1 under the validation-FPR threshold policy.

API prefix	AUC	Precision	Recall	F1
10 calls	0.9672	0.9494	0.8756	0.9111
25 calls	0.9813	0.9775	0.9016	0.9380
50 calls	0.9877	0.9780	0.9223	0.9493
100 calls	0.9905	0.9538	0.9637	0.9588
Full sequence	0.9902	0.9439	0.9585	0.9512

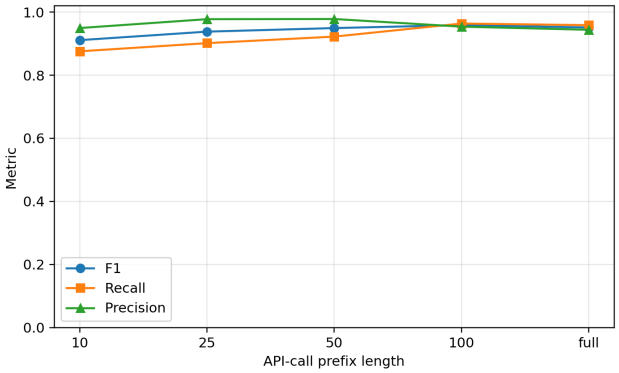


Fig. 5. MalBehavD-V1 prefix evaluation. The curve shows that useful discrimination is available before the full API-call sequence is observed.

split. The oracle threshold improves F1 only slightly, from 0.9512 to 0.9661. This small oracle gap is important: unlike BETH, where threshold transfer is fragile, MalBehavD-V1 has enough balanced validation signal for threshold selection to behave more stably.

Across 30 repeated stratified splits under the validation-FPR policy, mean F1 is 0.9530, median F1 is 0.9533, and the interquartile range is [0.9429, 0.9634]. Mean AUC is 0.9869 and mean AP is 0.9902. These repeated-split results show that the external result is not a single favorable split.

Table 11 and Figure 5 address the early-detection limitation. With only the first 10 API calls, the model already reaches F1 0.9111. Performance improves with longer prefixes, and the 100-call prefix slightly exceeds the full-sequence F1 under the validation-FPR policy. This does not prove real-time ransomware containment, but it shows how FAIR-X can be extended from full-trace classification to prefix-based behavioral detection.

Finally, supplementary permutation attribution on the fixed external test split reports API-specific and family-rate features derived only after final evaluation. They are not used to tune the model, preserving the separation between model selection and interpretation.

TABLE 12
BETH TabRF prefix evaluation under validation-FPR thresholding.

Prefix	AUC	Prec.	Rec.	F1
10 events	0.4443	0.6250	0.0413	0.0775
25 events	0.5553	0.9000	0.2231	0.3576
50 events	0.6678	0.8500	0.2810	0.4224
100 events	0.6414	0.7857	0.3636	0.4972
250 events	0.6572	0.9020	0.3802	0.5349
Full trace	0.7360	0.7500	0.6942	0.7210

12 Additional Limit-Lifting Audits on BETH

The external MalBehavD-V1 experiment tests whether the evaluation protocol transfers to a second behavioral schema. We also add five BETH-specific audits to reduce remaining ambiguity: feature attribution, prefix detection, cost-sensitive thresholding, feature-space stress testing, and host/temporal split robustness. These audits use the already selected TabRF model unless stated otherwise. They are not used to tune the main result.

Supplementary held-out permutation importance for TabRF shows that the dominant predictors are eventId histogram features, event entropy, process size, duration, event rate, and argument diversity. This supports the interpretation that TabRF is not relying on label annotations or ATT&CK-derived predictors; it is using observable behavioral composition and process-scale statistics.

Table 12 evaluates TabRF on prefixes of BETH process traces; the corresponding curve is supplementary. The validation-FPR threshold is reselected on the corresponding validation prefix. The result is materially weaker than full-trace classification: F1 is 0.0775 after 10 events, 0.3576 after 25 events, 0.5349 after 250 events, and 0.7210 only when the full process trace is available. This directly addresses the latency limitation. On BETH, the current full-process representation should not be claimed as an early containment detector.

Table 13 replaces a single F1 operating point with explicit cost assumptions. When false negatives and false positives have equal cost, the lowest empirical cost occurs at threshold 0.0274, with 21 false positives and 42 false negatives. When a false negative costs at least five times a false positive, the cost-minimizing threshold on this test split is zero, producing no false negatives but 77 false positives. This is not a deployable recommendation, because it uses test labels to illustrate cost sensitivity. It shows that response policy cannot be inferred from F1 alone.

Table 14 reports simple feature-space stress tests at the original validation-FPR threshold. Event-histogram dropout and malicious slowdown do not collapse the detector, but they change precision, recall, and ranking. These tests are not a substitute for malware-level evasion experiments; they

TABLE 13
BETH TabRF cost-sensitive threshold audit on the official test split. Thresholds are oracle cost minima and are reported only to expose cost sensitivity.

FN:FP	Thr.	FP	FN	F1
1:1	0.0274	21	42	0.7149
5:1	0.0000	77	0	0.7586
10:1	0.0000	77	0	0.7586
25:1	0.0000	77	0	0.7586
50:1	0.0000	77	0	0.7586

TABLE 14
BETH TabRF feature-space stress tests at the original validation-FPR threshold.

Stress test	AUC	Recall	F1
None	0.7360	0.6942	0.7210
10% event dropout	0.7334	0.7355	0.7177
25% event dropout	0.6852	0.8430	0.7312
50% event dropout	0.6437	0.8843	0.7405
Slowdown 0.50	0.7341	0.6612	0.6987
Slowdown 0.25	0.7488	0.6777	0.7100
Slowdown 0.10	0.7830	0.7521	0.7583

are a concrete lower-bound audit showing how the trained model reacts when behavioral composition or event timing is perturbed.

Finally, Table 15 evaluates group and temporal robustness by retraining TabRF on the enriched development pool under host-disjoint validation splits and a chronological 70/15/15 split, then evaluating once on the official test split. Four of ten host-disjoint validation attempts are skipped because one fold is single-class. Among the six valid host-disjoint folds, median F1 is 0.2739, with range [0.1678, 0.7188]. The chronological split reaches F1 0.4294. This audit partially lifts the split-design limitation by measuring it directly, but it does not eliminate it: positive examples are too concentrated across hosts for host-disjoint threshold selection to be stable.

13 Protocol Audit

13.1 Leakage Controls

The first audit question is whether the experiment leaks labels into the predictor. The pipeline avoids the obvious leakage path by excluding `sus` and `evil` from feature vectors. These columns are used only to define the process-level target. ATT&CK mappings are not used as predictors because BETH does not provide technique labels or documented mapping rules for them. The official test file is read during preprocessing to construct test features, but its labels are not used for fitting models, calibration, or threshold selection.

A subtler leakage risk is feature vocabulary construction. If the event vocabulary were fit on train, validation, and test jointly, test-only event identifiers would influence the feature space. The pipeline fits the event vocabulary on the enriched development pool and applies it to test. Unknown test events are handled without expanding the trained feature space. This is important because an event identifier appearing only in the test split could otherwise become a silent test-derived feature.

13.2 Threshold Fairness

The second audit question is whether all models receive equivalent threshold treatment. FAIR-X evaluates every model under the same validation-derived threshold policies. Under this rule, TabRF and RF-500 at the validation-FPR threshold reach F1 0.7210 and 0.7043, exceeding MetaRF at F1 0.6756. This demonstrates why threshold fairness is not a cosmetic concern; it can determine the interpretation of a hybrid detector.

The paper therefore reports two threshold policies for every model. Neither policy is perfect, but both are applied consistently. This creates a transparent table in which a reader can see how a model behaves under a false-positive constraint and under a validation-F1 objective.

13.3 Calibration Audit

The calibration fold contains six malicious processes. This is enough for scikit-learn logistic regression to fit Platt scaling, but it is not enough to trust probability calibration. The Platt-scaled scores are all close to the low development prevalence. The absolute thresholds reported in the supplementary material are correspondingly small. These numbers should not be interpreted as true posterior probabilities. They are only monotonic transformed scores.

The calibration audit motivates the decision to avoid probability-language in the final claims. We refer to "scores" and "thresholds" rather than calibrated risk probabilities. A deployment system would require a much larger calibration set from the target environment.

13.4 Ablation Completeness

The ablation is complete for the main scientific question because it separates score-only fusion, tabular learning, and tabular-plus-score learning. It does not exhaust all possible architectures. It does not test temporal convolution, transformer encoders, graph neural networks, process-tree aggregation, or host-level aggregation. That is acceptable because the goal is not to win BETH with every possible model; the goal is to test whether detector-score fusion adds value over a strong tabular baseline. The answer is negative.

13.5 Reproducibility Artifacts

The run writes partitioned data, fitted-model, score, metric, and figure artifacts. The added audits write cross-fitted thresholds, locked-threshold sensitivity, recurrent-capacity results, calibration, attribution, prefix, cost, stress, and split-robustness tables. Exact filenames and execution order are listed in the supplementary reproducibility checklist; generated CSV and JSON files are the authoritative numerical sources.

14 Failure Mode Analysis

14.1 Distribution Mismatch

The most important failure mode is distribution mismatch. The enriched development pool has a malicious rate near 0.5%, while the official test split has a malicious rate of 61.11%. A detector trained and thresholded under the development distribution will not necessarily preserve its FPR or

TABLE 15
BETH split-robustness audit. Host-disjoint results summarize six valid folds; four additional folds were single-class and skipped.

Split audit	AUC	Recall	F1
Main process split	0.7360	0.6942	0.7210
Host-disjoint median	0.5041	0.1860	0.2739
Host-disjoint range	0.4095–0.8300	0.0992–0.5702	0.1678–0.7188
Temporal 70/15/15	0.6865	0.2893	0.4294

recall on the test distribution. The validation-FPR threshold for TabRF produces 28 false positives on 77 benign test processes, corresponding to a test FPR of 36.36%. This is far above the 5% validation budget.

There are two possible interpretations. The pessimistic interpretation is that the validation fold is not representative enough for operational thresholding. The constructive interpretation is that BETH intentionally stresses out-of-distribution detection, and the failure to preserve FPR is itself a useful measurement. A practical endpoint system would need environment-specific validation or online threshold adaptation.

14.2 Representation Compression

ScoreRF performs worse than direct tabular baselines because detector scores compress high-dimensional behavior into a few scalar values. The Isolation Forest score summarizes isolation difficulty. The GRU-AE score summarizes reconstruction error. Both scores may discard which event types, durations, and argument patterns caused the anomaly. Tabular forests retain these details. Random Forest splits can directly exploit event-specific and scalar interactions.

This explains why MetaRF does not help. Adding compressed scores to the raw feature vector may introduce redundant or noisy variables. A tree ensemble can ignore unhelpful features in principle, but with very few positives it may still fit spurious interactions. The result is a small degradation relative to the strongest direct tabular baselines.

14.3 Validation Positive Scarcity

The fixed stratified validation fold contains four positives, so one changed decision moves validation recall by 25 percentage points. Section 10.6 reduces reliance on that fold by selecting 50 thresholds from repeated held-out folds containing five or six positives and locking their median before test evaluation. The resulting threshold range is narrow, but this procedure reuses the same 29 development positives and therefore does not create new independent evidence. Fine-grained threshold optimality remains unsupported; the defensible result is a stability and transfer audit.

14.4 Family-Specific Ambiguity

The evaluated data do not provide family-level ransomware labels. For this reason, the paper does not make a family-specific claim. It focuses on behavioral malicious-process detection under BETH distribution shift, which is the claim supported by the available labels.

14.5 Operational Cost Mismatch

F1 treats precision and recall symmetrically. Endpoint response does not. A false positive that suspends a hospital workstation, a domain controller, or an industrial host may be costly. A false negative that allows encryption may also be catastrophic. The right operating point depends on asset value, action reversibility, and analyst capacity. The paper reports F1 because it is standard, but it does not claim F1 is the only operational criterion.

15 Discussion

15.1 What the Experiment Supports

The experiment supports seven claims. First, BETH process-level detection is strongly affected by split design. The official train and validation files do not provide malicious process examples, so a supervised detector requires either supplementary development data or a different formulation. Second, the tabular process representation is highly competitive. It captures event diversity, temporal density, return-code behavior, and eventId composition. Third, thresholding policy is not a secondary implementation detail; it changes the apparent ranking of methods. Fourth, ATT&CK technique labels should be treated as predictive contributions only when supplied by validated telemetry rules or ground-truth annotations. Fifth, FAIR-X provides a compact way to audit hybrid behavioral detectors: a positive tabular-dominance gap and a large score-compression gap show that fusion has not earned its complexity on this split. Sixth, the same protocol can be applied to a second behavioral dataset with a different schema: on MalBehavD-V1, repeated splits, prefix evaluation, and held-out permutation attribution produce stable and interpretable evidence without oracle thresholding. Seventh, limit-lifting audits can turn vague deployment objections into measurable quantities: BETH prefix F1 remains weak before full-trace aggregation, cost-optimal thresholds depend strongly on the FN:FP ratio, and host-disjoint validation is unstable because positives are concentrated.

15.2 What the Experiment Does Not Support

The experiment does not support the statement that a hybrid AI pipeline improves over a strong tabular baseline. It also does not support family-specific ransomware attribution. The BETH dataset supports behavioral malicious-process and anomaly analysis, but it does not by itself certify family-specific generalization. The MalBehavD-V1 experiment strengthens protocol externality, not direct transfer of a BETH-trained model across incompatible feature schemas. Finally, the experiment does not validate automated response. The cost and prefix audits expose response-relevant

constraints, but containment still requires wall-clock latency, false-positive cost, rollback safety, endpoint integration, and human-in-the-loop evaluation.

15.3 On Negative Results in Security ML

Negative results are valuable in security machine learning because overly optimistic claims can lead to unsafe deployments. A detector that looks good only because a baseline used a weaker threshold policy is not an operational improvement. A semantic branch that depends on undocumented mapping assumptions may improve interpretability while reducing reproducibility. A calibrated score trained on six positives may look probabilistic but not behave as a reliable probability under deployment shift. Reporting these failures improves the field by narrowing the space of defensible claims.

15.4 The ExtraTrees Ranking–Decision Paradox

ExtraTrees is the clearest illustration of why FAIR-X separates ranking metrics from thresholded decisions. It obtains the best ranking metrics in the additional audit, with AUC 0.7973 and AP 0.8572, so it would be the apparent winner under a ranking-only comparison. Under the validation-FPR threshold, however, it produces only two false positives but misses 66 malicious processes, yielding recall 0.4545 and F1 0.6180. The model ranks many malicious processes above benign ones, but the validation-derived operating point is too conservative for the shifted official test distribution. This is not a contradiction; it is the exact distinction between score ordering and operational action. A paper reporting only AUC/AP would overstate ExtraTrees, while a paper reporting only F1 would hide its strong ranking behavior. FAIR-X requires both views and makes the threshold policy part of the scientific claim.

15.5 Operational Implications

FAIR-X has direct implications for endpoint and SOC evaluation practice. First, a behavioral detector should not be accepted for operational use solely because it has strong AUC or AP; the deployed threshold must be selected before final-test labels and must be audited under the target distribution. Second, threshold calibration should be environment-specific. The BETH audit shows that a 5% development-FPR objective can expand to more than 30% test FPR, so a global threshold learned in one host population may not be dependable for another host role, workload, or collection period.

Third, early response claims require prefix or wall-clock latency evidence. The full-trace TabRF result is useful for offline malicious-process classification, but the BETH prefix audit shows that the same representation is weak early in a process trace. A detector intended for containment should report detection quality at early event prefixes or time windows, not only after the complete trace is available. Fourth, response actions should be tied to reversibility and cost. When false-positive and false-negative costs differ, the preferred operating point can change sharply; automated process termination therefore requires stronger evidence than alerting or telemetry enrichment. FAIR-X does not solve deployment engineering, but it gives reviewers and practitioners a checklist for deciding whether a behavioral malware result supports ranking, alerting, or response claims.

15.6 Implications for Future Ransomware Benchmarks

Future behavioral endpoint-security benchmarks should provide:

- host-level and campaign-level split metadata;
- sufficient positive examples in development folds;
- family labels when family-specific claims are expected;
- event schemas with stable semantic documentation;
- predeclared threshold-selection protocols;
- external test sets for distribution-shift and schema-transfer evaluation.

Without these elements, it is difficult to distinguish robust detection from benchmark-specific thresholding.

16 Threats to Validity

16.1 Internal Validity

The implementation prevents direct label leakage by excluding `sus` and `evil` from predictor features. The test split is not used for training, calibration, or threshold selection. The main enriched development split is randomly stratified at process level, not split by campaign or time. Section 12 therefore adds host-disjoint and temporal-disjoint audits. These audits show that threshold representativeness remains fragile: several host-disjoint folds are single-class, and valid host-disjoint folds have much lower median F1 than the main process split.

16.2 Construct Validity

The process label is positive if any event in the process is labeled evil. This is a standard aggregation rule but may overstate process-level maliciousness when only a small subset of events is evil. Conversely, it may understate multi-process attacks if malicious behavior is distributed across parent and child processes. A graph-level representation could better capture process trees, but it would require a different experimental design.

16.3 External Validity

The BETH results are specific to BETH and to the process-level aggregation used here. They should not be generalized to all ransomware families or all endpoint platforms without a family-labeled endpoint dataset. The MalBehavD-V1 experiment validates the FAIR-X evaluation contract on an independent API-call sequence dataset, but it does not prove that the BETH-trained feature representation transfers across schemas. The title and claims are therefore framed around behavioral malware evaluation under distribution shift.

16.4 Statistical Validity

The official BETH test set has 198 processes. Bootstrap intervals quantify uncertainty but cannot create additional independent BETH evidence. The difference between the strongest tabular baselines and MetaRF is not large enough to justify a universal dominance claim. The safer conclusion is that MetaRF does not provide evidence of improvement over direct tabular learning in this setting. The MalBehavD-V1 repeated-split analysis reduces concern that the protocol is BETH-only, but it remains a separate dataset with a different observation schema. The host-disjoint audit also shows high variance because the positive examples are concentrated; therefore, split-robustness results should be read as stress evidence rather than as a stable deployment estimate.

17 Future Work and Claim Boundaries

The strongest claim supported by the experiment is methodological: threshold policy, baseline strength, and diagnostic gaps determine whether a hybrid pipeline appears useful. The empirical claim is narrower: on the BETH process-level configuration studied here, TabRF and RF-500 have higher F1 point estimates than MetaRF under the common validation-FPR policy, and detector-score fusion does not provide evidence of improvement. The family-level claim is deliberately absent because the evaluated data do not provide family-specific labels. ATT&CK mapping is treated as response-layer context unless a validated mapping procedure is introduced.

The repeated threshold audit removes dependence on one four-positive fold but cannot increase the number of independent positive processes. The next experimental step is therefore a larger schema-compatible endpoint dataset with host and time metadata, family labels, and enough positive processes to reserve independent calibration and test cohorts. The remaining deployment-grade steps are direct cross-environment transfer, family-labeled ransomware validation, malware-execution evasion tests, wall-clock detection delay, and response evaluation with rollback and analyst workload. These are new evidence requirements, not conclusions supplied by BETH or MalBehavD-V1.

18 Conclusion

We introduced FAIR-X, a threshold-transfer evaluation contract for defensible behavioral malware evaluation under distribution shift, and instantiated it as FAIR-BETH on BETH. The contract keeps ATT&CK mappings as response context unless validated technique labels are available, applies identical threshold policies, enforces baseline parity, reports uncertainty, and summarizes fusion utility through diagnostic gaps. Under the common validation-FPR policy, TabRF obtains the highest F1 point estimate, 0.7210, followed by RF-500 at 0.7043; their intervals overlap and MetaRF does not improve over either. The repeated threshold audit replaces one threshold estimate with 50 pre-test estimates over all 29 development positives. Their locked median yields F1 0.6964, while the development FPR budget expands to 32.47% on test, exposing non-transfer rather than concealing it.

The target-copy-free recurrent ablation shows that neither a 7.7-fold parameter increase nor an LSTM cell recovers useful BETH discrimination under next-event prediction; this is a bounded negative result, not a general rejection of sequence learning. The MalBehavD-V1 experiment shows that the evaluation contract can also be executed on another behavioral schema, with repeated-split median F1 0.9533. BETH prefix F1 remains 0.0775 after 10 events and reaches 0.7210 only at full trace, so the evaluated detector is not an early-containment system.

The central lesson is that behavioral aggregation and evaluation discipline matter more than architectural complexity in this setting. A hybrid architecture can be useful, but only if it improves over a strong baseline under the same threshold-selection protocol and under realistic distribution shift. On BETH, FAIR-X reports a positive tabular-dominance gap and a large score-compression gap, so the evidence favors a simpler tabular behavioral detector and a cautious interpretation of fusion, calibration, and automated response claims.

Acknowledgements

This work was supported by the authors' institutions. The authors used AI-assisted language and coding tools to improve readability, implementation checks, and reproducibility packaging. After using these tools, the authors reviewed and edited the content, verified the experimental outputs, and take full responsibility for the publication.

Declarations

Ethical Approval This study does not contain studies with human participants or animals performed by any of the authors. The experiments use publicly available cybersecurity datasets and offline evaluation scripts.

Competing Interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding This research received no external funding.

Author Contributions Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, and writing—original draft preparation: S.G.R.E. and U.J.A.N.; conceptualization, methodology, and writing—review and editing: S.A.E. and V.B.K.; formal analysis and investigation: S.W.M.T. and D.B.M.F.; data curation and validation: A.U.E.; supervision, project administration, and writing—review and editing: J.M.N. All authors read and approved the article.

Data Availability The BETH dataset is publicly available from the original dataset source cited in this article. MalBehavD-V1 is publicly available from the GitHub repository cited in this article. The paper reports only derived aggregate metrics, figures, and model-evaluation artifacts.

Code Availability The reproducibility package contains the FAIR-BETH pipeline, the external MalBehavD-V1 validation script, generated CSV result tables, and plotting code. The public repository is available at https://github.com/EkodeckStephane/fair-beth-malicious-process-detection.

REFERENCES

[1] M. Sikorski and A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. San Francisco, CA, USA: No Starch Press, 2012.

[2] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," in *2018 APWG Symposium on Electronic Crime Research (eCrime)*. IEEE, 2018, pp. 1–9.

[3] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[4] K. Highnam, K. Arulkumaran, Z. Hanif, and N. R. Jennings, "BETH dataset: Real cybersecurity data for anomaly detection research," in *Proceedings of the Workshop on Uncertainty and Robustness in Deep Learning*, ser. CEUR Workshop Proceedings, vol. 3095. CEUR-WS, 2021, pp. 1–14.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM)*. IEEE Computer Society, 2008, pp. 413–422.

[6] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL, 2014, pp. 1724–1734.

[7] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1285–1298.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1321–1330.

[11] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[13] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.

[14] S. O. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, pp. 6679–6687, 2021.

[15] MITRE Corporation. (2023) MITRE ATT&CK Framework. Accessed: 2026-05-08. [Online]. Available: https://attack.mitre.org/

[16] B. E. Strom, A. Applebaum, D. P. Miller, B. E. Strom, R. Korban, and K. Nickels, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep., 2018. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_May_2018.pdf

[17] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[18] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

[19] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, "API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques," *Journal of Network and Computer Applications*, vol. 218, p. 103704, 2023.

[20] P. Maniriho, "MalBehavD-V1: API call sequences extracted from malware and benign windows executables," https://github.com/mpasco/MalbehavD-V1, 2023, accessed: 2026-06-03.